CS324 Distributed Systems

Assignment 1 Part 1: Report

Group Members:

Joshua Dovale — Student ID: [S11230279]
Pranay Singh— Student ID: [S11232318]
Aveshal Anand — Student ID: [S11230749]

Team Responsibilities and Contribution Breakdown

Member	Responsibilities	Contribution %
Joshua Dovale	Bootstrap Node design & implementation; worker registration & bidirectional network join logic;, report work, Debugging.	100%
Pranay Singh	Leader election algorithm design & implementation (message flooding, duplicate prevention, JAC-based winner resolution); coordinator broadcast; job-splitting logic, report work	100%
Aveshal Anand	Multithreaded job execution (MAX/PRIMESUM/PRIMECOUNT); coordinator job dispatch & result aggregation; Swing client GUI with manual entry and CSV loading, report work.	100%

Question 1: The Importance of the Bootstrap Node

The Bootstrap Node is the entry point that allows a new worker to join the network. Java RMI needs a host and a port before a lookup can succeed so a new worker cannot find peers by itself. When a worker starts it registers with the Bootstrap Node. The Bootstrap Node gives the worker an ID. Adds it to a live registry. The worker then asks for the list of peers. Connects to one randomly chosen existing worker as a neighbour. This satisfies the rule that a joining worker must be randomly linked to a peer.

Without a Bootstrap Node, the usual alternative is a seed list. The joiner receives one or more fixed worker addresses. Asks a seed for contacts. That still needs some knowledge of least one address. Multicast or mDNS discovery can remove the need, for knowledge. It works only on a local network and adds complexity that this assignment does not need.

The Bootstrap Node does not participate in leader election or job processing. It only handles membership and discovery allowing the peer-to-peer worker layer to form.

Question 2: Does the Bootstrap Node Make the System Centralised?

centralised?

Only for membership. Every new worker must contact the Bootstrap Node to register and receive an ID so the Bootstrap Node is a point of failure for joining. Workers that are already connected do not need the Bootstrap Node to keep running. Election and job processing remain decentralised. Workers send ELECTION and COORDINATOR messages to their neighbours report candidacy to the initiator. Choose the JAC (the highest ID when tied). Clients send work to the elected coordinator. The coordinator divides the range. Calls executeJob on workers via RMI. None of this traffic passes through the Bootstrap Node. If the Bootstrap Node fails after the network has formed the existing workers can still elect a leader and process jobs among themselves. Thus the design is hybrid: a directory for discovery and decentralised control for computation as the assignment tests.

Question 3: Preventing Duplicate Election Message Processing

Preventing election message processing
Election uses flooding. The initiator sends an ElectionMessage, which has an electionId made from workerId and timestamp to its neighbours. When a worker sees that ID for the time it reports its JAC back to the initiator and forwards the message to its neighbours. Because the network is unstructured and may have cycles the same message can arrive times. Each worker keeps a thread‑safe seenElections set (ConcurrentHashMap.newKeySet()). The first line of receiveElectionMessage is a seenElections.add(electionId). If the ID is already present the method returns immediately. Does not report candidacy or again. The same approach is used for CoordinatorMessage with seenCoordinations. Without this floods would not finish in a graph: workers would keep re‑forwarding. The network would fill with duplicate traffic. The seen‑set ensures each worker processes. Forwards a given election once. Forwarding is performed by a floodPool so the RMI call can return quickly and avoid nested RMI deadlocks on the connection.

Question 4: Threads and Thread-Safety Mechanisms

Threads and thread-safety

Concurrency. At the coordinator submitJob splits the range across workers and starts one dispatch thread per worker so the RMI executeJob calls run in parallel; the coordinator then joins those threads. Combines results. At each worker executeJob further splits its chunk across a fixed ExecutorService (up to four threads). Combines the Future results. That meets the requirement for job execution with Java threads.

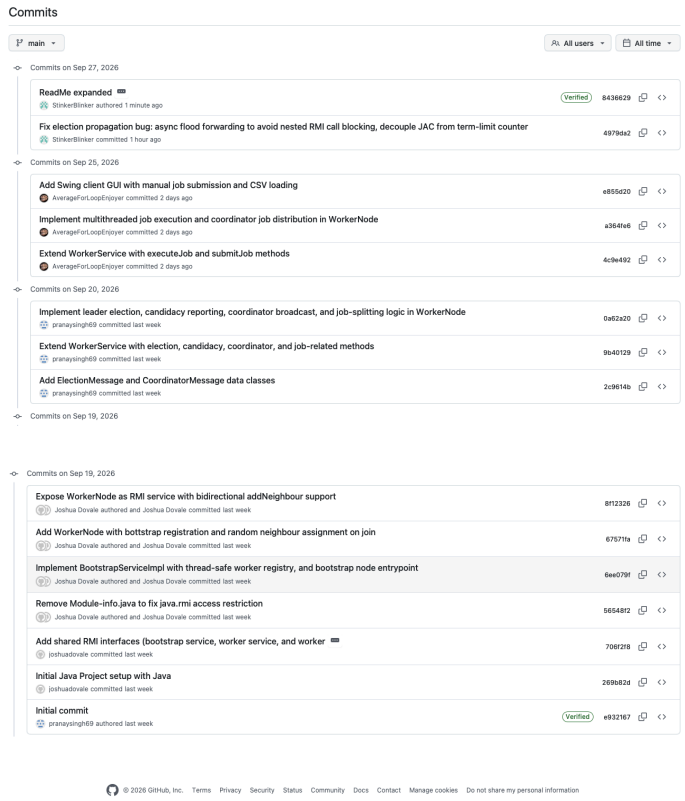
Election and coordinator announcement use a daemon floodPool so receive handlers return immediately while forwards and candidacy reports run asynchronously. That avoids holding an RMI call stack while calling peers back (which caused missed COORDINATOR messages under elections in the design).

Thread-safety. Shared state is protected as follows:

* ConcurrentHashMap / newKeySet() for seen elections, seen coordinations and candidacy maps
* AtomicInteger for JAC. Jobs-this-term
* AtomicBoolean for election-in-progress
* CopyOnWriteArrayList for the neighbour list (read-heavy rare writes)
* volatile, for coordinatorId. The last accepted election timestamp
* A short synchronized block when applying a coordinator so timestamp and ID update together

Appendix A: Git Commit Log

The following is the complete commit history for the project repository (cs324-A1)



NOTE: Github Contributor: AverageForLoopEnjoyer - Aveshal Anand, Joshua Dovale, pranaysingh69 - Pranay Singh, StinkerBlinker - Joshua Dovale.

Appendix B: Use of AI Tools

The Usage of AI was done through Claude and Perplexity following the selective AI Usage policy

- AI was used in assisting in the creation of the FrontEnd, GUI.
- AI was used in locating resources and studies to aid in the understanding of the assignment.
- AI was used in providing examples and test cases towards the system.
- AI was used in checking for grammatical errors within the report to ensure proper grammar fluency.

References

Oracle. (2024). Trail: RMI. In The Java Tutorials. Oracle Corporation.

https://docs.oracle.com/javase/tutorial/rmi/index.html

Oracle. (2024). Interface Remote (Java SE 21 & JDK 21). Java Platform, Standard Edition API Specification.

https://docs.oracle.com/en/java/javase/21/docs/api/java.rmi/java/rmi/Remote.html

Oracle. (2024). Class LocateRegistry (Java SE 21 & JDK 21). Java Platform, Standard Edition API Specification.

https://docs.oracle.com/en/java/javase/21/docs/api/java.rmi/java/rmi/registry/LocateRegistry.html

Oracle. (2024). Class UnicastRemoteObject (Java SE 21 & JDK 21). Java Platform, Standard Edition API Specification.

https://docs.oracle.com/en/java/javase/21/docs/api/java.rmi/java/rmi/server/UnicastRemoteObject.html

Oracle. (2024). Class ConcurrentHashMap (Java SE 21 & JDK 21). Java Platform, Standard Edition API Specification.

https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html

Oracle. (2024). Class AtomicInteger (Java SE 21 & JDK 21). Java Platform, Standard Edition API Specification.

https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/atomic/AtomicInteger.html

Oracle. (2024). Class CopyOnWriteArrayList (Java SE 21 & JDK 21). Java Platform, Standard Edition API Specification.

https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/CopyOnWriteArrayList.html

Oracle. (2024). Interface ExecutorService (Java SE 21 & JDK 21). Java Platform, Standard Edition API Specification.

https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/ExecutorService.html